



Hermes

⚙ Status In progress

J'adore l'idée. Et je vais te dire un truc : **ne considère pas ce projet comme "une app", mais comme un petit système d'exploitation distribué entre ton Mac et ton iPhone.** C'est beaucoup plus motivant à concevoir de cette façon.

Combien de temps ?

Si tu connais bien le C, Python, les sockets, les processus et que tu es à l'aise en terminal :

- **Jour 1** : découverte de Swift + premier démon macOS + compilation en CLI.
- **Jour 2** : communication TCP/TLS entre le Mac et un client de test.
- **Jour 3** : première application iPhone qui communique avec le démon.
- **Jour 4** : authentification, signatures, stockage sécurisé des clés.
- **Jour 5** : premières commandes (lock, battery, screenshot, notification...).
- **Jour 6-7** : nettoyage, architecture, tests, amélioration de la sécurité.

En une semaine intensive, tu peux avoir un prototype très propre. En quelques jours, tu auras déjà quelque chose de fonctionnel.

Cahier des charges — Projet "Hermes"

*(Je lui donne un nom de code : **Hermes**, le messenger.)*

1. Objectif

Développer un système personnel, entièrement natif, permettant à un Mac et un iPhone de communiquer de manière sécurisée, sans dépendance à des applications tierces pour les fonctionnalités principales.

Le projet est avant tout un exercice d'ingénierie système, de développement natif Apple, de sécurité et de programmation réseau.

Le système doit être modulaire, extensible et maintenable.

2. Contraintes Fonctionnelles

- contrôle bidirectionnel Mac ↔ iPhone
 - communication temps réel
 - faible latence (<100 ms sur réseau local)
 - sécurité forte
 - fonctionnement automatique
 - aucune interface graphique obligatoire côté Mac
 - architecture extensible
-

Techniques

Langages :

- Swift (principal)
- éventuellement un peu de C si nécessaire
- Python uniquement pour scripts de test

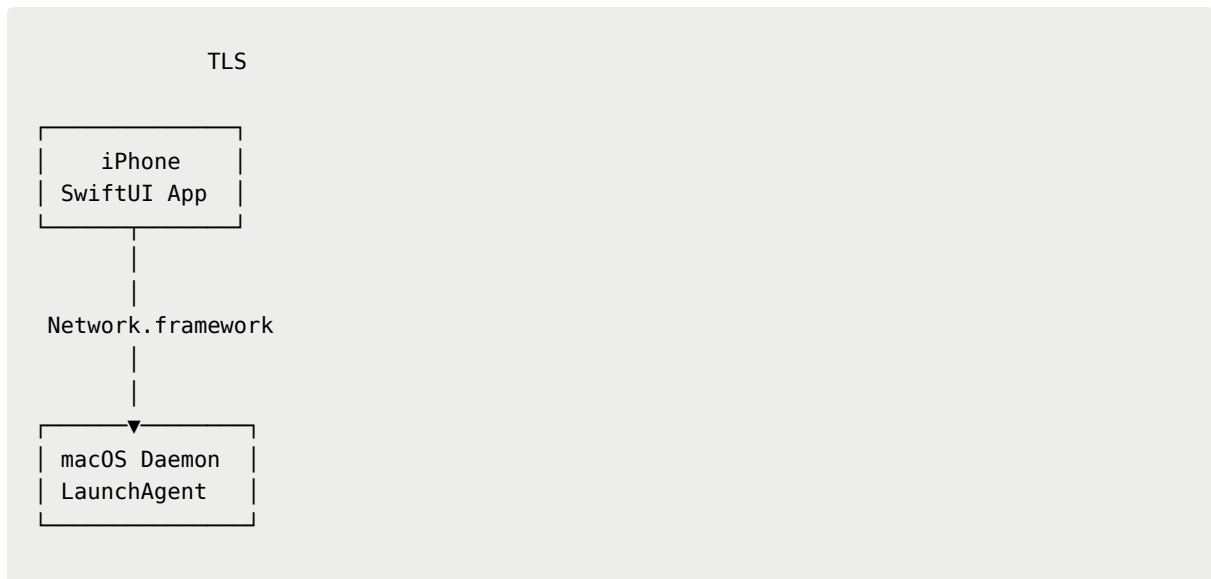
Plateformes :

- macOS
- iOS

Outils :

- VS Code
 - iTerm2
 - Git
 - Xcode uniquement lorsque nécessaire (signature, simulateur, provisioning)
-

3. Architecture



Le Mac est le serveur.

L'iPhone est le client.

Les rôles pourront évoluer ensuite.

4. Processus macOS

Créer un démon lancé automatiquement via LaunchAgent.

Responsabilités :

- écouter un port TCP
- gérer TLS
- authentifier les clients
- écouter les événements système
- exécuter les commandes
- envoyer des événements

Toujours actif.

Aucune interface graphique.

5. Application iPhone

Application SwiftUI très simple.

Responsabilités :

- connexion au Mac
 - authentification
 - interface de commandes
 - réception des notifications
 - affichage des événements
-

6. Communication

Utiliser :

Network.framework

Éviter les sockets BSD brutes sauf à des fins pédagogiques.

Connexion persistante.

Possibilité d'utiliser :

- TCP
 - TLS
 - WebSocket ensuite
-

7. Protocole

Créer un protocole propriétaire.

Exemple :

```
Header

magic
version
type
length
```

```
timestamp
nonce

payload

signature
```

Chaque paquet doit être sérialisé proprement.

Aucun JSON dans la première version.

Préférer un protocole binaire.

8. Cryptographie

Utiliser exclusivement les API Apple.

CryptoKit

Chaque appareil possède :

- clé privée
- clé publique

Première connexion :

- échange de clés
- validation
- stockage dans le Trousseau (Keychain)

Chaque paquet :

- signé
- horodaté
- nonce aléatoire

Protection contre :

- replay
 - modification
 - usurpation
-

9. Événements Mac

Le démon doit détecter :

- verrouillage
- déverrouillage
- ouverture de session
- fermeture de session
- veille
- réveil
- changement réseau
- batterie
- alimentation
- montage de disque
- capture d'écran éventuelle
- lancement d'applications (optionnel)

Chaque événement peut être envoyé immédiatement à l'iPhone.

10. Commandes Mac

À implémenter progressivement :

LOCK

SLEEP

SHUTDOWN

REBOOT

OPEN_APP

RUN_COMMAND

SCREENSHOT

BATTERY
WIFI
CPU
RAM
DISKS
VOLUME
MUTE
CLIPBOARD
FILES
NOTIFICATION

Toutes les commandes devront demander une authentification valide.

11. Fonctionnalités iPhone

L'application pourra :

- afficher les notifications
 - consulter l'état du Mac
 - envoyer des commandes
 - consulter les captures
 - recevoir des événements
 - afficher les logs
-

12. Fonctionnalités futures

Synchronisation :

- presse-papiers
- fichiers
- notes

Découverte réseau :

- Bonjour (mDNS)

Connexion distante :

- VPN
- Tailscale
- WireGuard

Widgets iOS

Live Activities

Apple Watch

Apple Vision Pro

13. Journalisation

Créer un système de logs.

Chaque action est enregistrée.

Exemple :

```
2026-06-29 13:40:18  
  
COMMAND : LOCK  
  
REMOTE : iPhone  
  
RESULT : SUCCESS  
  
LATENCY : 31 ms
```

Prévoir plusieurs niveaux :

- INFO
 - DEBUG
 - WARNING
 - ERROR
-

14. Structure du projet

```
Hermes/  
  
  daemon/  
  
  protocol/  
  
  crypto/  
  
  commands/  
  
  events/  
  
  network/  
  
  shared/  
  
  ios/  
  
  tests/  
  
  docs/
```

Créer un package Swift partagé pour éviter la duplication du code entre macOS et iOS.

15. Objectifs pédagogiques

Découvrir :

- Swift moderne
- Swift Concurrency (`async/await` , acteurs)
- Package Manager
- Network.framework
- CryptoKit
- Keychain
- LaunchAgent
- Unified Logging (`os.Logger`)

- Notifications système
 - Signature de code
 - Développement iOS
 - SwiftUI (juste assez pour l'interface)
 - Architecture modulaire
 - Protocoles binaires
 - Cryptographie appliquée
-

16. Philosophie du projet

Le projet ne cherche pas à reproduire un logiciel existant. Son but est de construire une infrastructure personnelle, sécurisée et élégante, permettant de comprendre en profondeur les mécanismes de communication entre appareils Apple.

Les priorités sont :

1. sécurité avant tout ;
 2. simplicité de l'architecture ;
 3. code lisible et modulaire ;
 4. utilisation maximale des API natives Apple ;
 5. minimisation des dépendances externes ;
 6. apprentissage par la pratique.
-

Une dernière suggestion

J'ajouterais un objectif ambitieux qui donne une vraie identité au projet : **viser une qualité "Apple"**. Cela signifie :

- zéro dépendance externe si les frameworks Apple suffisent ;
- API internes propres et documentées ;
- tests unitaires dès le début ;
- protocole versionné pour pouvoir évoluer sans tout casser ;

- journalisation soignée ;
- gestion rigoureuse des erreurs ;
- sécurité pensée dès la conception, pas ajoutée après coup.

Si tu construis ce projet avec cet état d'esprit, tu apprendras Swift, mais aussi une façon de concevoir des logiciels robustes qui te servira bien au-delà de ce projet. J'aurais vraiment plaisir à t'accompagner dessus, car il couvre un ensemble de sujets passionnants sans tomber dans la complexité artificielle.